Document Parse

Upstage Document Parse는 문서를 자동으로 HTML로 변환하도록 설계된 강력한 AI 모델입니다. 이 모델은 단락, 표, 이미지 등과 같은 레이아웃 요소를 감지하여 문서의 구조를 결정합니다. 그런 다음 API는 읽기 순서에 따라 요소를 정렬하고, 마지막으로 문서를 HTML로 변환합니다.

사용 가능한 모델

Alias	모델명	설명
document-parse	document-parse-250404	- 폼 형태 문서 지원 강화 - 회전된 문서 처리 능력 향상 - 여러 페이지에 걸친 테이블 처리 지원 - 긴 이미지에 대한 처리 지원 - ocr=auto 일때 PDF 인식이 실패하면 자동으로 OCR 수행 - hwp, hwpx 파일 타입 지원

모델 출력 결과 이해하기

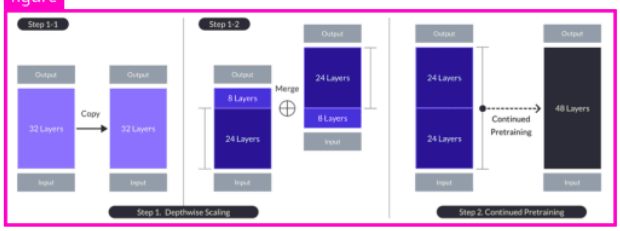
레이아웃 카테고리와 HTML 태그

Upstage Document Parse는 입력 문서에서 다양한 레이아웃 요소를 식별하고 HTML을 생성하여 입력 내용을 디지털화 형식으로 나타냅니다. 레이아웃 범주는 미리 정의된 집합이며, 아래 예제 그림에서 모델이 탐지할 수 있는 모든 유형의 레이아웃 범주에 대한 자세한 예를 볼 수 있습니다.

header Solar 1.7B

header Upstage AI

figure



caption

Figure 1: Depth up-scaling for the case with $n = 32$, $s = 48$, and $m = 8$. Depth up-scaling is achieved through a dual-stage process of depthwise scaling followed by continued pretraining.

paragraph for wider access and application of these models by researchers and developers globally.

2 Depth Up-Scaling

heading1

paragraph To efficiently scale-up LLMs, we aim to utilize pre-trained weights of base models to scale up to larger LLMs (Komatsuzaki et al., 2022). While existing methods such as Komatsuzaki et al. (2022) use MoE (Shazeer et al., 2017) to scale-up the model architecture, we opt for a different depthwise scaling strategy inspired by Tan and Le (2019). We then continually pretrain the scaled model as just scaling the model without further pretraining degrades the performance.

paragraph **Base model.** Any n-layer transformer architecture can be used but we select the 32-layer Llama 2 architecture as our base model. We initialize the Llama 2 architecture with pretrained weights from Mistral 7B, as it is one of the top performers compatible with the Llama 2 architecture. By adopting the Llama 2 architecture for our base model, we aim to leverage the vast pool of community resources while introducing novel modifications to further enhance its capabilities.

paragraph **Depthwise scaling.** From the base model with n layers, we set the target layer count s for the scaled model, which is largely dictated by the available hardware.

paragraph With the above, the depthwise scaling process is as follows. The base model with n layers is duplicated for subsequent modification. Then, we remove the final m layers from the original model and the initial m layers from its duplicate, thus forming two distinct models with $n - m$ layers. These two models are concatenated to form a scaled model with $s = 2 \cdot (n - m)$ layers. Note that $n = 32$ from our base model and we set $s = 48$ considering our hardware constraints and the efficiency of the scaled model, *i.e.,* fitting between 7 and 13 billion parameters. Naturally, this leads to the removal of $m = 8$ layers. The depthwise scaling process with $n = 32$, $s = 48$, and $m = 8$ is depicted in 'Step 1: Depthwise Scaling' of Fig. 1.

paragraph

paragraph We note that a method in the community that also scale the model in the same manner [2] as 'Step 1: Depthwise Scaling' of Fig. 1 has been concurrently developed.

paragraph **Continued pretraining.** The performance of the depthwise scaled model initially drops below that of the base LLM. Thus, we additionally apply the continued pretraining step as shown in 'Step 2: Continued Pretraining' of Fig. 1. Experimentally, we observe rapid performance recovery of the scaled model during continued pretraining, a phenomenon also observed in Komatsuzaki et al. (2022). We consider that the particular way of depthwise scaling has isolated the heterogeneity in the scaled model which allowed for this fast performance recovery.

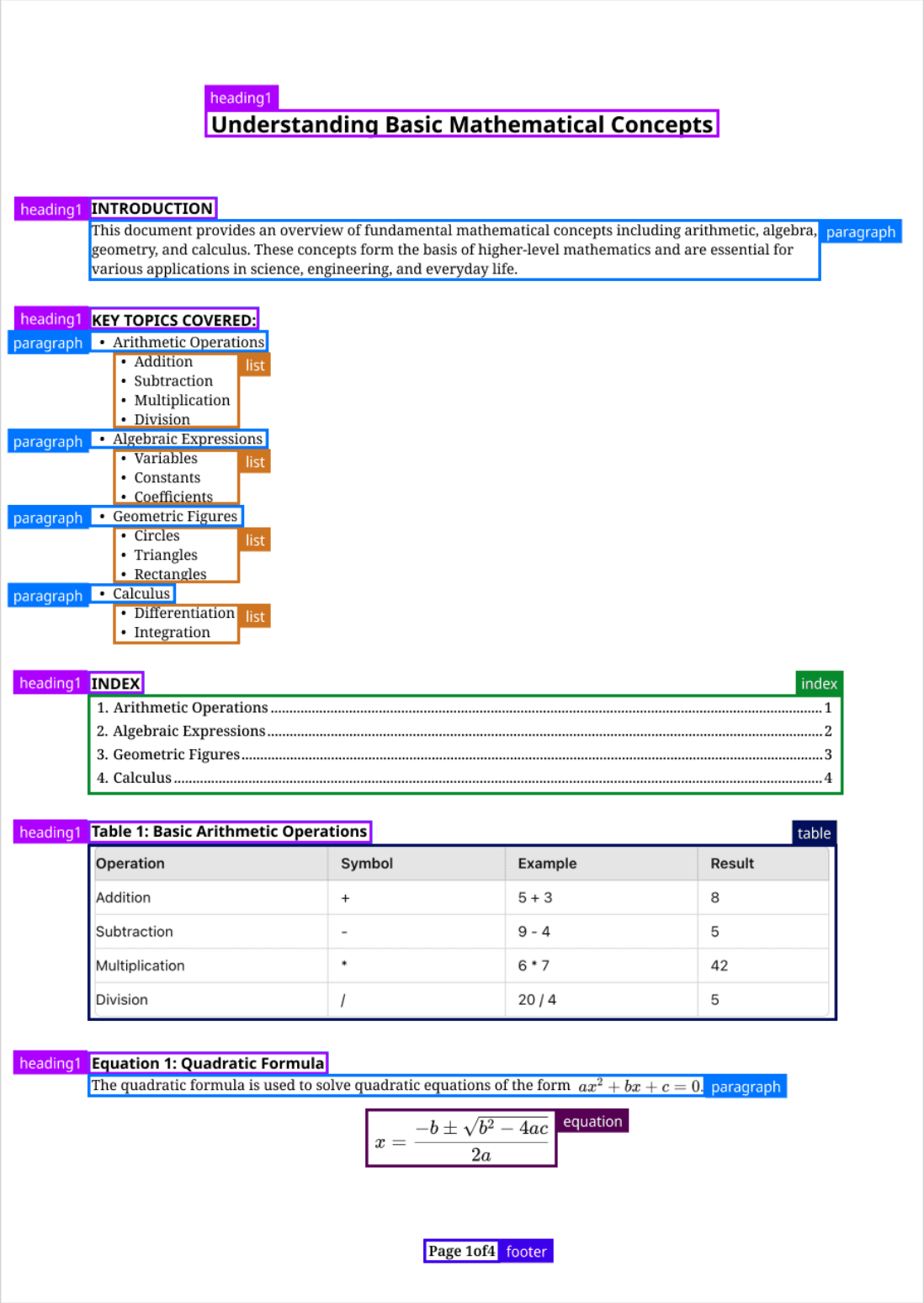
paragraph Delving deeper into the heterogeneity of the scaled model, a simpler alternative to depthwise scaling could be to just repeat its layers once more, *i.e.,* from n to $2n$ layers. Then, the 'layer distance', or the difference in the layer indices in the base model, is only bigger than 1 where layers n and $n + 1$ are connected, *i.e.,* at the seam.

paragraph However, this results in maximum layer distance at the seam, which may be too significant of a discrepancy for continued pretraining to quickly resolve. Instead, depthwise scaling sacrifices the $2m$ middle layers, thereby reducing the discrepancy at the seam and making it easier for continued

[2] https://huggingface.co/Undi95/Mistral-11B-v0.1

footnote

2 footer



The table below explains layout categories and corresponding HTML tags. It uses the specific HTML tag when it represents the same layout category in both printed and HTML documents. If there is no suitable HTML tag for the layout category, it uses a <p> tag with a data-category attribute to explain the layout category detected by the model.

아래 표는 레이아웃 범주와 이에 대응되는 HTML 태그를 설명합니다. 인쇄된 문서와 HTML 문서 모두 동일 레이아웃 범주를 나타낼 때 특정 HTML 태그를 사용합니다. 만약 레이아웃 범주에 적합한 HTML

태그가 없는 경우, 모델이 감지한 레이아웃 범주를 설명하기 위해 data-category 속성을 가진 태그를 사용합니다.

Category	HTML
table	<table> .. </table>
figure	<figure><img> .. </img></figure>
chart	<figure><img data-category="chart"> .. </img></figure>
heading1	<h1>... </h1>
header	<header> .. </header>
footer	<footer> .. </footer>
caption	<caption> .. </caption>
paragraph	..
equation	..
list	..
index	..
footnote	

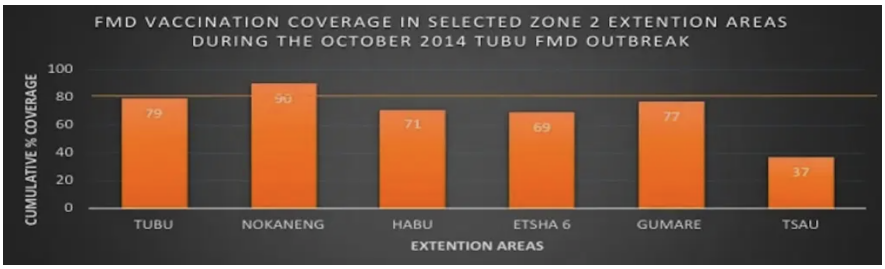
차트 인식 출력 활용하기

문서 내의 차트는 유용한 정보를 담고 있지만, 이전에는 이미지로 파싱되어 검색, QA 또는 RAG와 같은 작업에 활용하기 어려웠습니다. 이 문제를 해결하기 위해 멀티 모달 LLM을 사용할 수 있지만, 텍스트 기반 LLM에 비해 더 비싸고 더 복잡한 파이프라인으로 이어지는 경향이 있습니다.

새로 추가된 차트 인식 기능을 활용하면 문서 내 바, 선, 원형 차트가 인식되어 표 형태로 변환됩니다. 차트에서 수치 데이터를 추출해 표로 만들면 기존의 텍스트 기반 파이프라인에 원활하게 통합되어 차트 정보를 효과적으로 활용할 수 있습니다.

새로운 Document Parse 제품은 다음 형식으로 차트 범주 출력에 대한 인식 결과를 제공합니다.

아래는 예시 이미지에 대한 인식 결과입니다.



```
<figure id='2' data-category='chart'>

  <!— Provides coordination information about the chart —>
  <img data-coord="top-left:(118,157); bottom-right:(900,393)" />

  <!— Provides metadata about the chart. Each item is separated by a <p> tag. —>
  <figcaption>
    <p>Chart Title: FMD VACCINATION COVERAGE IN SELECTED ZONE 2 EXTENTION AREAS DURING THE
OCTOBER 2014 TUBU FMD OUTBREAK</p>
    <p>X-Axis: EXTENTION AREAS</p>
    <p>Y-Axis: Cumulative % Coverage</p>
    <p>Chart Type: bar</p>
  </figcaption>

  <!— Converts the recognized chart results into a table format. —>
  <table>
    <tr>
      <th></th>
      <th>TUBU</th>
      <th>NOKANENG</th>
      <th>HABU</th>
      <th>ETSHA 6</th>
      <th>GUMARE</th>
      <th>TSAU</th>
    </tr>
    <tr>
      <td>item_01</td>
      <td>79%</td>
      <td>90%</td>
      <td>71%</td>
      <td>69%</td>
      <td>77%</td>
      <td>37%</td>
    </tr>
  </table>
</figure>
```

<img> - 차트에 대한 좌표 정보를 제공합니다.

<figcaption> - Contains chart metadata, with each item separated by a <p> tag.

차트 메타데이터를 포함하며, 각 항목은 <p> 태그로 구분됩니다.

1. 차트 제목(선택 사항) - 차트의 제목
2. x축(선택 사항) - x축의 단위
3. y축(선택 사항) - y축의 단위
4. 차트 유형 - 막대, 선 또는 파이 차트로 식별됨

<table> - 인식된 차트 데이터를 테이블 형식으로 나타냅니다.

다음은 API 응답 내용입니다.

```
{
  "category": "chart",
  "content": {
    "html": "<figure id='4' data-category='chart'><img data-coord=\"top-left:(175,1305);
bottom-right:(785,1784)\" /><figcaption><p>Chart Type: bar</p></figcaption><table><tr><th></
th><th>Nuclear</th><th>Renewables</th><th>Hydro</th><th>Natural gas</th><th>Coal</th><th>Oil</
th></tr><tr><td>item_01</td><td>4%</td><td>4%</td><td>7%</td><td>24%</td><td>27%</td><td>34%</
td></tr></table></figure>",
    "markdown": "- Chart Type: bar\n|  | Nuclear | Renewables | Hydro | Natural gas | Coal |
Oil |\n| --- | --- | --- | --- | --- | --- | --- |\n| item_01 | 4% | 4% | 7% | 24% | 27% | 34%
|\n",
    "text": "Chart Type: bar \nNuclear Renewables HydroNatural gas Coal Oil\n item_01 4% 4% 7%
24% 27% 34%"
  },
  "id": 4,
  "page": 1
}
```


- **content.html**: 차트가 인식되면 <figcaption>과 <table> 요소가 <figure data-category='chart'> 태그 내에 추가됩니다.
- **content.markdown**: HTML을 Markdown 형식으로 변환한 결과입니다.
- **content.text**: HTML에서 텍스트만 추출한 결과입니다.

차트 인식은 내부적으로 스코어링 메커니즘을 사용하여 인식의 성공을 결정합니다. 만약 인식이 실패하면, 차트는 '그림'으로 분류되어 다음 형식으로 처리됩니다.

- **content.html**: 차트 인식이 성공적으로 완료되면, OCR 결과는 <figure data-category='chart'> 내에 있는 <img> 태그의 alt 속성에 추가됩니다.
- **content.markdown**: OCR 결과 입니다.
- **content.text**: OCR 결과 입니다.


```
{
  "category": "figure",
  "content": {
    "html": "<figure id='4' data-category='chart'><img style='font-size:20px' alt=\"Nuclear 4%
\nRenewables 4%\nHydro 7%\nNatural gas 24%\nCoal 27%\nOil 34%\" data-coord=\"top-left:
(175,1306); bottom-right:(785,1783)\" /></figure>",
    "markdown": "Nuclear 4%\nRenewables 4%\nHydro 7%\nNatural gas 24%\nCoal 27%\nOil 34%",
    "text": "Nuclear 4%\nRenewables 4%\nHydro 7%\nNatural gas 24%\nCoal 27%\nOil 34%"
  },
  "id": 4,
  "page": 1
}
```


수실 출력 사용하기

수실 출력에 대해 모델은 <p data-category="equation"> 태그 내부에 LaTex 형식의 인식된 텍스트를 제공하여 MathJax과 같은 대중적인 수식 렌더링 엔진을 사용하여 HTML 문서에서 올바른 표시를

할 수 있습니다. 수식 이미지의 출력 예제는 아래와 같습니다.

$$a_n=\sum_{k=1}^{n}\frac{2k+1}{1^2+2^2+3^2+\cdots+k^2}$$


```
{
  "category": "equation",
  "content": {
      "html": "<p id='3' data-category='equation'>$$a_{n}=\\sum_{k=1}^{n}{\\frac{2k+1}{\
\,1^{2}+2^{2}+3^{2}+\\cdots+k^{2}}}$$</p>",
      "markdown": "$$a_{n}=\\sum_{k=1}^{n}{\\frac{2k+1}{\\,1^{2}+2^{2}+3^{2}+\\cdots+k^{2}}}$
$,
      "text": "n 2k+1 \nan  \nk=1 12+22+32 + · · +k2"
  },
}
```


응답에서 content.html과 content.markdown 필드는 인식된 수식을 LaTex 형식으로 포함하고 있지만, content.text 필드는 OCR 결과 텍스트를 포함하고 있습니다. 현재 OCR 모델은 수식 인식을 지원하지 않기 때문에 content.text 필드는 정확한 수식 텍스트를 포함하지 않을 수 있습니다. 수식 카테고리에 대해서는 content.text 필드의 OCR 결과가 하위 호환성을 위해 제공되었습니다.

사용자는 HTML 파일에서 content.html 필드 값을 제대로 렌더링하는 데 어려움을 겪을 수 있습니다. 이는 JSON 형식의 API 응답이 \ 문자를 이스케이프하기 때문입니다. 이 문제를 해결하려면 JavaScript를 사용하여 HTML을 이스케이프 해제할 수 있습니다. 아래의 예제 코드는 MathJax 라이브러리를 임포트하고 API 응답을 이스케이프 해제하여 HTML 파일에서 수식을 올바르게 렌더링하는 방법을 보여줍니다.

```
<body>
    <div id="equation" /> <!— placeholder for equation —>
    <script id="MathJax-script" async src="https://cdn.jsdelivr.net/npm/mathjax@3/es5/tex-mml-
chtml.js"></script>
    <script type="text/javascript">
        <!— put the API response to innerHTML field —>
        document.getElementById('equation').innerHTML = "<p id='5' data-category='equation'>$
$f(x)=a_{0}+\\sum_{n=1}^{\\infty}\\left(a_{n}\\cos{\\frac{n\\pi x}{L}}+b_{n}\\sin{\\frac{n\\pi
x}{L}}\\right)$$</p>"
    </script>
<body>
```

사용자는 요청 본문의 base64_encoding 필드에 ["equation"]을 지정하여 원본 문서에서 방정식의 base64 인코딩 이미지를 얻을 수도 있습니다.

소수점 네자리 좌표 형식

사용자는 Document Parse 모델을 사용하여 입력 문서에서 경계 상자를 그리거나 요소를 자를 때 상

대 좌표를 사용할 수 있습니다. 절대 위치는 각 항목의 coordinates 필드에서 x 및 y값을 요청한 이미지의 너비와 높이로 곱하여 계산할 수 있습니다.

```
"coordinates": [
 {
   "x": 0.0276,
   "y": 0.0178
 },
 {
   "x": 0.1755,
   "y": 0.0178
 },
 {
   "x": 0.1755,
   "y": 0.0641
 },
 {
   "x": 0.0276,
   "y": 0.0641
 }
],
```

왜 상대 좌표일까요? 이는 사용자가 이전 버전의 응답 결과를 바운딩 박스를 이미지 아닌 입력 문서에 렌더링하려고 시도했을 때 어려움을 겪었기 때문입니다. Document Parse 엔진은 시각적 특징 감지 및 인식을 위해 입력 문서를 내부적으로 이미지로 변환합니다. 변환 중에 이미지 품질과 속도 사이의 균형을 찾기 위해 다양한 방법을 사용하며, 입력 문서 크기에 따라 다른 DPI 설정을 선택하게 됩니다. 그래서 내부적으로 변환된 이미지의 크기 정보가 없으면 사용자는 자신의 문서에 바운딩 박스를 정확하게 그릴 수 없었습니다. 이 문제를 해결하기 위해 Document Parse 엔진은 각 바운딩 박스 포인트의 상대적 위치를 제공하여 너비-높이 비율이 유지되는 한 모든 크기의 변환된 이미지에 대해서 활용될 수 있도록 합니다.

요구 사항

예제 코드

요청

```
# pip install requests

import requests
filename = "YOUR_FILE_NAME"         # ex: ./image.png

url = "http://<NODE_IP>:8080/v1/document-parse"

files = {"document": open(filename, "rb")}
response = requests.post(url, files=files)

print(response.json())
```

응답

```
{
  "api": "2.0",
  "content": {
    "html": "<h1 id='0' style='font-size:22px'>INVOICE</h1>\n<h1 id='1' style='font-
size:20px'>Company<br>Upstage</h1>\n<br><h1 id='2' style='font-size:18px'>Invoice ID</
h1>\n<br><h1 id='3' style='font-size:14px'>휴 INV-AJ355548</h1>\n<h1 id='4' style='font-
size:18px'>Invoice Date</h1>\n<br><h1 id='5' style='font-size:18px'>9/7/1992</h1>\n<h1 id='6'
style='font-size:16px'>Mamo<br>Lucy Park</h1>\n<h1 id='7' style='font-size:18px'>Address</
h1>\n<br><h1 id='8' style='font-size:16px'>7 Pepper Wood Street, 130 Stone
Comer<br>Terrace<br>Wilkes Barre, Pennsylvania, 18768<br>United States</h1>\n<h1 id='9'
style='font-size:16px'>Email</h1>\n<br><h1 id='10' style='font-
size:16px'>Ikitchenman0@arizona.edu</h1>\n<br><h1 id='11' style='font-size:20px'>Service
Details Form</h1>\n<h1 id='12' style='font-size:16px'>Name<br>Sung Kim</h1>\n<h1 id='13'
style='font-size:16px'>260 'ess<br>Gwangovolungang:co 338, Gyeongg do.<br>Sanghyeon-dong, Sui-
gu<br>Yongin-si, South Korea</h1>\n<h1 id='14' style='font-size:18px'>Additional Request</
h1>\n<br><p id='15' data-category='paragraph' style='font-size:14px'>Vivamus vestibulum
sagittis sapien. Cum sociis natoque<br>penatibus 항목 magnis dfs parturient montes, nascetur
ridiculus<br>mus.</p>\n<h1 id='16' style='font-size:14px'>TERMS AND CONDITIONS</h1>\n<p
id='17' data-category='list' style='font-size:14px'>L TM Seir that not be lable 1층 the Buyer
drectly indirectly for any loun or damage sufflered by 전액 Buyer<br>2. The 별 www. the
product for ore 과 관한 from the date 설 shipment.<br>3. Any ourchase order received by ~ sele
- be interpreted 추가 accepting the offer Ma the 18% offer writing The buyer may<br>purchase
15 The offer My the Terms and Conditions the Seller included The offer</p>",
    "markdown": "",
    "text": ""
  },
  "elements": [
    {
      "category": "heading1",
      "content": {
        "html": "<h1 id='0' style='font-size:22px'>INVOICE</h1>",
        "markdown": "",
        "text": ""
      },
      "coordinates": [
        {
          "x": 0.0648,
          "y": 0.0517
        },
        {
          "x": 0.2405,
          "y": 0.0517
        },
        {
          "x": 0.2405,
          "y": 0.0953
        },
        {
          "x": 0.0648,
          "y": 0.0953
        }
      ],
      "id": 0,
      "page": 1
    },
    {
      "category": "heading1",
      "content": {
```

```
        "html": "<h1 id='1' style='font-size:20px'>Company<br>Upstage</h1>",
        "markdown": "",
        "text": ""
      },
      "coordinates": [
        {
          "x": 0.0657,
          "y": 0.2651
        },
        {
          "x": 0.1606,
          "y": 0.2651
        },
        {
          "x": 0.1606,
          "y": 0.3168
        },
        {
          "x": 0.0657,
          "y": 0.3168
        }
      ],
      "id": 1,
      "page": 1
    },
    ... 중략 ...
    {
      "category": "paragraph",
      "content": {
        "html": "<br><p id='15' data-category='paragraph' style='font-size:14px'>Vivamus
vestibulum sagittis sapien. Cum sociis natoque<br>penatibus 항목 magnis dfs parturient montes,
nascetur ridiculus<br>mus.</p>",
        "markdown": "",
        "text": ""
      },
      "coordinates": [
        {
          "x": 0.4191,
          "y": 0.6684
        },
        {
          "x": 0.9132,
          "y": 0.6684
        },
        {
          "x": 0.9132,
          "y": 0.7332
        },
        {
          "x": 0.4191,
          "y": 0.7332
        }
      ],
      "id": 15,
      "page": 1
    },
    {
      "category": "heading1",
```

```
      "content": {
        "html": "<h1 id='16' style='font-size:14px'>TERMS AND CONDITIONS</h1>",
        "markdown": "",
        "text": ""
      },
      "coordinates": [
        {
          "x": 0.0649,
          "y": 0.8303
        },
        {
          "x": 0.2506,
          "y": 0.8303
        },
        {
          "x": 0.2506,
          "y": 0.8523
        },
        {
          "x": 0.0649,
          "y": 0.8523
        }
      ],
      "id": 16,
      "page": 1
    },
    {
      "category": "list",
      "content": {
        "html": "<p id='17' data-category='list' style='font-size:14px'>L TM Seir that not be
lable 1층 the Buyer drectly indirectly for any loun or damage sufflered by 전액 Buyer<br>2.
The 별 www. the product for ore 과 관한 from the date 설 shipment.<br>3. Any ourchase order
received by ~ sele - be interpreted 추가 accepting the offer Ma the 18% offer writing The
buyer may<br>purchase 15 The offer My the Terms and Conditions the Seller included The offer</
p>",
        "markdown": "",
        "text": ""
      },
      "coordinates": [
        {
          "x": 0.0679,
          "y": 0.8717
        },
        {
          "x": 0.9261,
          "y": 0.8717
        },
        {
          "x": 0.9261,
          "y": 0.9558
        },
        {
          "x": 0.0679,
          "y": 0.9558
        }
      ],
      "id": 17,
      "page": 1
```

```
    }
  ],
  "model": "document-parse-240910",
  "usage": {
    "pages": 1
  }
}
```

API 레퍼런스

```
POST /v1/document-parse
```

요청 Body

document | required | file

처리할 문서 파일입니다. 지원되는 파일 형식은 요구사항에 나열되어 있습니다.

---

ocr | optional | string

문서 레이아웃 감지 전 OCR 추론을 수행할지 여부를 나타내는 문자열 값입니다. 가능한 값은 auto와 force 중 하나입니다. force로 설정하면 파일이 항상 이미지로 변환되고, 레이아웃 감지 전에 OCR이 수행됩니다. auto로 설정하면 OCR이 이미지 문서에만 적용됩니다. 이미지가 아닌 파일(예: PDF, DOCX)의 경우, 문서 레이아웃 및 텍스트 감지는 문서가 디지털-본(digital-born)일 때만 작동합니다. 즉, 드래그하여 텍스트를 직접 선택할 수 있어야 합니다. 기본값은 auto입니다.

---

coordinates boolean

각 레이아웃 요소의 경계 상자 좌표를 반환할지 여부를 나타내는 불리언 값입니다. 기본값은 true입니다.

---

output_formats string

각 레이아웃 요소의 출력 형식을 나타내는 문자열 값입니다. 가능한 값은 text, html 및 markdown입니다. 기본값은 ["html"] 입니다.

---

model string

추론에 사용되는 모델을 나타내는 문자열 값입니다. 기본적으로 API는 사용자가 특정 버전을 제공하지 않으면 최신 모델 버전을 선택합니다. 사용자는 별칭 (alias) 을 사용하여 모델 버전을 참조할 수도 있습니다. 사용 가능한 모델은 사용 가능한 모델 섹션에서 찾을 수 있습니다.

---

base64_encoding string

base64 인코딩된 문자열로 제공해야 하는 레이아웃 범주를 나타내는 문자열 값입니다. 모든 범주 이름은 레이아웃 카테고리와 HTML 태그 섹션에서 찾을 수 있습니다. 이 기능은 사용자가 원본 문서 이미지에서 레이아웃 요소를 자르고 자체 목적으로 저장하고 사용하려는 경우에 유용합니다. 예를 들어 사용자는 입력 문서의 모든 테이블에 대한 이미지 base64 인코딩을 추출할 수 있습니다. 모든 레이아웃 범주를 지정할 수 있습니다.

응답

기능 개요

- **알 수 없는 문자:** OCR 모델에서 인식되지 못한 문자는 "알 수 없는 문자"로 간주되며, 문자 ☒로 표시됩니다.
- **응답 시간:** 최대 200단어의 표준 문서는 대략 3초가 소요됩니다. 더 긴 문서는 최대 수십 초가 소요될 수 있습니다.
- **타임아웃:** 모든 요청에 대해 서버 측에서 5분 타임아웃이 있습니다.

응답 값

api string

현재 사용 중인 API 버전을 나타내는 문자열입니다. 주요 버전 증가(major version)는 하위 호환되지 않는 업데이트를 나타내며, 마이너 버전 증가(minor version)는 하위 호환되는 업데이트를 나타냅니다.

---

model string

사용되는 모델의 버전을 나타내는 문자열 입니다.

---

elements list

인식된 레이아웃 요소의 리스트로, 각 객체는 텍스트, HTML, 바운딩 박스에 대한 정보를 포함합니다.

---

elements[].id integer

레이아웃 요소의 고유 식별자

---

elements[].category string

주어진 레이아웃 요소의 범주

---

elements[].page integer

요소의 페이지 번호입니다. 1부터 시작합니다.

---

elements[].content.text string

요소 내의 텍스트를 나타내는 문자열입니다. 주로 요소에 대한 OCR 결과를 제공합니다.

---

elements[].content.html string

요소를 HTML 형식의 텍스트로 나타내는 문자열입니다.

---

elements[].content.markdown string

요소를 마크다운 형식의 텍스트로 나타내는 문자열입니다. heading1 , table, list 이외의 경우 text 필드 값과 동일합니다.

---

elements[].coordinates list

요소의 경계를 정의하는 네 개의 (x, y) 좌표가 포함된 목록입니다. 각 값은 0에서 1 사이의 4자리 소수점 값으로 입력 문서에서의 상대적 위치입니다. 픽셀 단위의 절대 위치를 얻으려면 사용자가 x 및 y를 각각 이미지 문서의 너비와 높이와 곱하면 됩니다.

---

elements[].base64_encoding string

레이아웃 요소의 좌표로 자른 이미지의 base64 인코딩된 문자열 값.

---

content.text string

페이지의 추출된 텍스트를 모두 연결하여 생성하는 문서의 전체 텍스트를 나타내는 문자열 입니다.

---

content.html string

페이지의 추출된 HTML 문자열을 모두 연결하여 생성된 문서의 전체 HTML 을 나타내는 문자열 입니다.

---

content.markdown string

페이지의 추출된 마크다운 문자열을 모두 연결하여 생성된 문서의 전체 마크다운을 나타내는 문자

열 입니다.

---

usage.pages integer

입력 파일에서 처리된 페이지의 수.

제품 사양

기본 사양

HW 기본 사양

HW 사양은 기본 솔라박스 플랫폼을 위한 시스템 사양과 그 위에서 동작하는 각 AI 모델을 사용하기 위한 사양으로 나뉩니다.

솔라박스 플랫폼 최소 사양

제품명	제품 버전	CPU	GPU (VRAM)	RAM	DISK
Solar Box	solarbox-2.0.0	16 cores	-	16GB	200GB

AI 모델별 필요 최소 사양

모델명	양자화 여부	파라미터 크기	context length	CPU	GPU (VRAM)	RAM	DISK	암호화 지원	Compute Cap.
solar-pro2 (2.0.0)	-	31B	64K	16 cores	80GB x 2	64GB	100GB+	Y	≥ 9.0 (H100)
solar-pro2 (2.0.0)	Yes (w4a16)	31B	64K	16 cores	80GB	64GB	100GB+	Y	≥ 9.0 (H100)
solar-pro (1.2.0)	Yes (w4a16)	22B	32K	16 cores	40GB	64GB	100GB+	Y	≥ 8.9 (L40s, H100)
solar-mini (1.3.0)	Yes (w4a16)	10.7B	32K	16 cores	16GB	64GB	100GB+	Y	≥ 8.9 (L40s, H100)
document--parse		-	-	8 cores	12GB	16GB	100GB	Y	≥ 8.0 (A10, A100, L40s)
Embeddings	No	10.7B	4K	16 cores	26GB	64GB	100GB+	N	≥ 8.0 (A100)

모델명	양자화 여부	파라미터 크기	context length	CPU	GPU (VRAM)	RAM	DISK	암호화 지원	Compute Cap.
KIE models	-	-	-	8 cores	3~10GB	16GB	50GB	N	≥ 8.0 (A10, A100, L40s)
OCR	-	-	-	8 cores	7.5GB	16GB	100GB	N	≥ 8.0 (A10, A100, L40s)

AWS instance 사양

Solar-Pro2

- FP16
  - p5.48xlarge
- 양자화 버전
  - p5.48xlarge

Solar-Mini

- FP16
  - g4dn.12xlarge
  - g5.12xlarge
  - g6.12xlarge
  - p3.8xlarge
  - p4d.24xlarge
  - p5.48xlarge
- 양자화 버전
  - g5.4xlarge
  - g6.4xlarge

- p5.48xlarge

Document Parse and KIE models

- g5.2xlarge
- g6.2xlarge
- p3.2xlarge

제품 설치

Prerequisite

System Requirements

모델	시스템	버전
공통	OS	≥ Ubuntu server 22.04 LTS
		≥ RHEL 8.6 (RHEL 9.4 Recommended)
	nvidia-driver	≥ 550.90.07
	cuda	≥ 12.4
	nvidia-container-toolkit	≥ 1.15.0-1
	Docker - Engine	≥ 24.0.5
	Docker - Compose	≥ 2.17.2

Setting up on Red Hat Enterprise Linux

솔라박스 v2.0 이상에서 RHEL 8.6 버전과 9.4 버전에서 테스트 되었으며 다음과 같은 환경 설정이 필요합니다.

필요 개발 패키지 설치

개발에 필요한 각종 패키지를 설치합니다.

```
sudo dnf -y groupinstall "Development Tools"
sudo dnf install -y elfutils-libelf-devel libglvnd-devel
```

Nvidia default kernel driver disable 및 cgroup을 v2로 세팅

nvidia default kernel driver는 추후 설치할 nvidia driver와 충돌이 나기 때문에 비활성화합니다.

podman container에 대해, cpu와 memory limit을 거는 기능을 위해 cgroup v2로 설정합니다.

```
sudo grubby --update-kernel=ALL --args="rd.driver.blacklist=nouveau modprobe.blacklist=nouveau
systemd.unified_cgroup_hierarchy=1 cgroup_no_v1=all"
```

Grub 적용

위에서 업데이트한 파일을 grub 파일에 적용합니다.

```
# 결과에 Not booted with EFI 가 포함되어 있으면 legacy, 아니면 EFI
bootctl status

# legacy Booting 모드 일 경우
sudo grub2-mkconfig -o /boot/grub2/grub.cfg
# EFI Booting 모드 일 경우
sudo grub2-mkconfig -o /boot/efi/EFI/redhat/grub.cfg
```

Boot mode를 Text Mode로 변경

현재 시스템이 gui가 아닌 cui 기반으로 동작하도록 설정합니다.

```
sudo systemctl set-default multi-user.target
sudo systemctl isolate multi-user.target
```

Selinux 비활성화

Selinux의 각종 정책과 충돌하여 어플리케이션이 이상 동작하는 것을 방지하고자, Selinux를 비활성화합니다.

```
sudo sed -i 's/^SELINUX=enforcing/SELINUX=disabled/;' /etc/selinux/config
```

리부팅

위의 내용들을 적용하기 위해 재부팅합니다. 위의 설정이 잘 적용되었는지 확인합니다.

```
sudo reboot

# v2이어야 합니다.
mount | grep cgroup

# 아무 것도 없어야 합니다.
lsmod | grep nouveau
```

Nvidia Driver 설치

설치되어 있는 gpu와 운영체제에 맞게 nvidia driver를 설치합니다.

```
# 예시
wget https://us.download.nvidia.com/XFree86/Linux-x86_64/550.142/NVIDIA-Linux-
x86_64-550.142.run
sudo bash NVIDIA-*
```

Nvidia Container Toolkit 설치

Podman container가 gpu를 물고 뜰 수 있도록 nvidia-container-toolkit을 설치합니다.

```
curl -s -L https://nvidia.github.io/libnvidia-container/stable/rpm/nvidia-container-
toolkit.repo | \
  sudo tee /etc/yum.repos.d/nvidia-container-toolkit.repo
sudo yum install -y nvidia-container-toolkit
```

Podman 설치

각종 서비스를 container 환경으로 띄울 수 있는 podman을 설치합니다.

```
sudo dnf groupinstall "Container Management"
```

Cgroup controller setting

각종 서비스가 cgroup v2 기반으로, cpu나 memory limit을 걸 수 있도록 설정합니다.

```
sudo mkdir -p /etc/systemd/system/user@.service.d/
sudo tee /etc/systemd/system/user@.service.d/delegate.conf > /dev/null << EOF
[Service]
Delegate=memory pids cpu cpuset
EOF
sudo reboot
sudo systemctl daemon-reexec

# cpu cpuset memory pid 있는지 확인
cat "/sys/fs/cgroup/user.slice/user-$(id -u).slice/user@$(id -u).service/cgroup.controllers"
cat "/sys/fs/cgroup/user.slice/user-$(id -u).slice/user@$(id -u).service/
cgroup.subtree_control"

# 안되어 있다면 아래와 같이 설정
echo "+cpu +cpuset" >> "/sys/fs/cgroup/user.slice/user-$(id -u).slice/user@$(id -u).service/
cgroup.subtree_control"

# cgroupControllers에 cpuset, cpu, memory, pids 가 있는지 확인합니다.
podman info
```

Fuse-overlayfs 활성화

기본 container overlay file system인 overlayfs는 container layer 로드시 critical한 버그가 존재합니다. 대안으로 fuse-overlayfs를 사용합니다.

```
mkdir -p ~/.config/containers/
tee ~/.config/containers/storage.conf > /dev/null << EOF
[storage]
driver = "overlay"

[storage.options.overlay]
mount_program = "/usr/bin/fuse-overlayfs"
EOF
```

Nvidia toolkit cgroup 설정

Nvidia-container에 대해서도 cgroup을 적용해서 각종 리소스 제한이 동작하도록 변경합니다.

```
sudo sed -i 's/^#no-cgroups = false/no-cgroups = true/;' /etc/nvidia-container-runtime/
config.toml
```

User를 journald 그룹에 추가

유저를 journald 그룹에 추가하여 podman에서도 로그 관련 operation이 권한 문제 없도록 설정합니다.

```
sudo usermod -aG systemd-journal $USER
```

Logout해도 세션 유지되도록 설정

사용자가 ssh 등 세션에서 로그아웃해도 세션이 유지되어 서비스가 계속 동작하도록 설정합니다.

```
# https://github.com/containers/podman/issues/16784
loginctl enable-linger $USER
```

Nvidia gpu 커널 모듈 로드

```
# https://forums.developer.nvidia.com/t/no-cuda-capable-device-is-detected/41756/6
# https://github.com/pytorch/pytorch/issues/49081
nvidia-modprobe -c 0 -u
```

Nvidia hook 파일 작성

Podman 컨테이너가 기본적으로 nvidia container toolkit을 이용해서 기동하도록 hook 을 설정합니다.

```
sudo tee /etc/containers/oci/hooks.d/oci-nvidia-hook.json > /dev/null << EOF
{
    "version": "1.0.0",
    "hook": {
        "path": "/usr/bin/nvidia-container-toolkit",
        "args": ["nvidia-container-toolkit", "prestart"],
        "env": [
            "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
        ]
    },
    "when": {
        "always": true,
        "commands": [".*"]
    },
    "stages": ["prestart"]
}
EOF

sudo mkdir -p /usr/share/containers/oci/hooks.d/
sudo ln -sf /etc/containers/oci/hooks.d/oci-nvidia-hook.json /usr/share/containers/oci/hooks.d/
oci-nvidia-hook.json
```

Nvidia container toolkit gpu 설정 세팅

Nvidia container toolkit이 현재 gpu 설정을 읽어와 cdi 구성을 생성합니다.

```
sudo nvidia-ctk cdi generate --output=/etc/cdi/nvidia.yaml

# device 확인
```

```
nvidia-ctk cdi list
```

Podman-docker 패키지를 설치

Podman을 사용하더라도 docker와 동일한 인터페이스로 이용할 수 있도록 podman-docker 패키지를 설치합니다.

```
sudo dnf install -y podman-docker
```

Standalone docker-compose 설치

여러 개의 컨테이너를 한번에 관리할 수 있도록 docker-compose를 설치합니다.

```
#설치 파일 다운로드
sudo curl -SL https://github.com/docker/compose/releases/download/v2.15.1/docker-compose-linux-
x86_64 -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
```

Podman socket 시작

Docker compose가 동작하기 위해서는 docker과 유사하게 podman daemon을 enable시켜야 합니다.

```
systemctl --user start podman.socket
ls $XDG_RUNTIME_DIR/podman/podman.sock
```

환경변수 세팅

Podman socket과 관련된 환경변수를 설정하고 현재 세션에 적용합니다.

```
echo 'export DOCKER_HOST=unix://$XDG_RUNTIME_DIR/podman/podman.sock' >> ~/.bashrc
source ~/.bashrc
```

동작 확인

Nvidia-toolkit-container가 제대로 동작하고, cgroup이 제대로 동작하는지 확인합니다. 아래 명령어가 오류 없이 잘 실행되면 성공입니다.

```
docker run --rm --cpus 1 -ti nvidia/cuda:11.0.3-base-ubuntu20.04 nvidia-smi
```

노드 구성

솔라 박스는 여러 서버 노드에 모델과 컴포넌트들을 분산하여 구동할 수 있습니다. 하지만 솔라박스는 고수준의 MLOps 를 위한 클러스터링을 지원하는 플랫폼은 아니기 때문에 정적인 설정 파일을 통해서 여러 노드에 모델과 컴포넌트를 분산합니다.

설정 파일

여러 노드에 모델과 서비스를 분산하기 위해서 가장 먼저 수행할 업무는 configuration.yaml 을 작성하는 것입니다. 이 작업은 필요한 파일을 업스테이지 서버에서 다운로드 받은 직후 수행해야 하며 다운로드 받은 전체 도커 이미지를 각 노드에 어떻게 분산하여 구동할지를 작성하는 과정입니다. 파일은 bundled 디렉토리와 같은 위치인 최상위 디렉토리에 위치시킵니다.

configuration.yaml 파일의 예제와 스키마는 다음과 같습니다.

```yaml
name: "solarbox-test"
version: "1.0.0"
description: ""
nodes:
  - name: "NODE_1_NAME"
    ip: "NODE_IP"
    cpus: 64
    memory: "256GB"
    gpus: 1
    gpuMemory: "24GB"
    deployments:
      - catalog: "ocr-2.2.1"
        gpuDevice: "0"
      - catalog: "ocr-2.2.1"
        gpuDevice: "1"
      - catalog: "earth-0.0.2"
      - catalog: "agent-podman-2.26.3"
      - catalog: "dcgm-exporter-2.3.4-2.6.4-ubuntu20.04"
      - catalog: "node-exporter-v1.8.2"
```

최상위 필드

- name (string): 솔라박스 이름을 정의합니다.
- version (string): 솔라박스의 버전을 명시합니다.
- description (string): 솔라박스에 대한 설명을 제공합니다.
- nodes (array): 클러스터 내 개별 노드를 정의하는 배열입니다. 각 요소는 하나의 노드를 나타내며, 다음과 같은 속성을 가집니다.

node 필드

- name (string): 노드의 고유 식별자입니다. 일반적으로 호스트 네임을 사용합니다.
- ip (string): 노드의 IP 주소입니다.
- cpus (integer): 노드에서 사용 가능한 논리 CPU 개수입니다. (nproc 명령어 실행 결과)
- memory (string): 노드의 총 메모리 크기입니다. (free -h 명령어 실행 결과)
- gpus (integer): 노드에서 사용 가능한 GPU 개수입니다.
- gpuMemory (string): GPU 한 장의 VRAM 값입니다.

- deployments (array): 해당 노드에서 실행될 배포 항목을 정의하는 배열입니다.

deployment 필드

- catalog (string): 배포할 애플리케이션 또는 서비스의 이름과 버전입니다.
- gpuDevice (string, optional): GPU가 필요한 모델의 경우, 할당할 GPU의 인덱스를 지정합니다. 0 부터 시작합니다.

오퍼레이터

솔라박스에 포함된 오퍼레이터는 설정 파일을 기반으로 하여 각 노드에 필요한 번들을 생성합니다. 다음과 같은 명령어를 통해 오퍼레이터를 실행시킬 수 있습니다.

```
$> ./bundled/scripts/02_operate.sh
```

스크립트를 실행하면 오퍼레이터는 다운로드 받은 각 카탈로그 항목들의 config 파일에 실제 노드 정보를 기입합니다. 또한 각 카탈로그별로 존재하는 docker compose 명령을 노드 별로 병합하여 각 노드에 사용할 최종 docker compose 설정 파일을 생성합니다. 이 후 생성된 파일과 함께 노드를 세팅하는데 필요한 파일들을 한데 모아 outputs/<NODE_NAME>/deploy-bundle.tar.gz 형태로 압축을 수행합니다.

설정 파일 검증

오퍼레이터가 번들 파일을 생성하기 전에 작성된 configuration.yaml 파일에 대한 기본적인 검증을 수행합니다.

- deployment 아래 각 카탈로그 아이템이 해당 솔라박스에 존재하는 버전인지 검사합니다.
- deployment 아래 각 카탈로그 아이템이 의존하는 필수 의존성 카탈로그 아이템들이 함께 기입되어 있는지 확인합니다.
- deployment 아래 각 카탈로그 아이템의 시스템 리소스 요구사항을 기반으로 전체 카탈로그 아이템을 노드의 cpus / memory / gpus 를 이용하여 모두 구동할 수 있는지 체크합니다.

노드간 파일 복사

01_download_files.sh 스크립트 실행을 통해서 솔라박스에 필요한 모든 도커이미지와 필요한 파일들을 다운로드 받았다면 다운로드 받을 파일을 구동한 전체 노드에 복사합니다. 노드간 파일 복사에 대해서는 솔라박스에서 지원하는 기능은 없기 때문에 scp 등을 이용하여 설치 엔지니어가 직접 파일을 복사해야 합니다. 다운로드 받은 파일은 downloads 디렉토리 아래 존재하며 downloads 디렉토리를 모든 노드에 복사하는것을 권장합니다. 또한 위에서 오퍼레이터에서 생성된 각 노드별 deploy-bundle.tar.gz 파일도 함께 복사하여 압축을 해제 합니다.

디렉토리 구성 예제:

```
/home/solarbox
├__downloads
├__bundled
├__deploy-bundle.tar.gz
```

파일 복사가 완료 되었다면 각 노드에 필요한 정보는 모두 준비되었기 때문에 다운로드된 파일을 이용하여 실제 솔라박스를 띄우는 작업을 수행할 준비가 되었습니다.

수동 설정

솔라 박스의 일부 기능을 이용하기 위해서는 불가피하게 설치 후 일부 설정들이 수동으로 이루어 져야 합니다. 아래는 수동 설정이 필요한 항목 들에 대한 설명입니다.

비동기 추론을 위한 스토리지 설정

비동기 추론을 위한 이미지 파일 저장 및 결과 저장을 위해, S3 호환 오브젝트 스토리지 역할을 하는 SeaweedFS를 사용합니다. SeaweedFS 카탈로그는 “solarbox”라는 bucket을 제공합니다. 다음은 solarbox 버킷에 TTL(Time To Live)을 설정하는 방법이고 반드시 설정하는 것을 권장합니다.

스토리지 TTL 설정하는 법

TTL 포맷은 <숫자><단위> 형식으로 설정할 수 있습니다. 예를 들어:

- 30m → 30분
- 1h → 1시간
- 7d → 7일
- 1w → 1주
- 30d → 30일

다음 명령어를 사용해 solarbox 버킷에 대해 TTL을 설정할 수 있습니다.

예시: 30일 TTL 설정

```
docker exec -it <seaweedfs-master-container-name> weed shell
fs.configure -locationPrefix=/buckets/solarbox/ -ttl 30d -apply
exit
```

- <seaweedfs-master-container-name>: SeaweedFS master 컨테이너 이름

기본 기능

로깅

일반 로깅

API 인 Earth 와 Agent 는 지정된 경로에 로그를 저장하고 로테이션 합니다. 경우에 따라서는 파일 로깅을 끄고 표준 출력으로만 로깅하도록 설정할 수도 있습니다.

Earth 의 기본 로깅 설정은 다음과 같습니다.

```
logging:
    stdout:
      enable: true
    file:
      enable: true
      path: "/var/log/earth.log"
      maxSize: 100000
      maxMinutes: 1440
      maxAge: 0
      maxBackups: 0
      compress: false
```

기본 설정으로 로그는 표준 출력과 파일에 모두 쓰여지며 파일 생성된지 1일이 넘거나 100MB 가 넘어가는 경우 로테이션 됩니다. 로깅 파일은 컨테이너 내부의 /var/log/earth.log 파일에 저장되며 해당 경로가 호스트 머신의 earth-0.1.0/logs 경로로 마운트 됩니다.

LLM 로깅

LLM 모델을 서빙할 때는 감사(Audit) 목적의 **입·출력 로깅** 기능을 추가로 활성화할 수 있습니다. 로그 파일은 **Earth 컨테이너 내부** /var/log/llm 경로에 저장되며, 호스트 머신에서는 mounts/earth-0.1.0/logs/llm 디렉토리에 마운트됩니다. 해당 로그에는 **LLM 호출 시 사용된 프롬프트와 파라미터, 그리고 모델 응답 전체**가 기록됩니다. 기본값(enable)은 false이며, 필요 시 true로 변경하여 사용합니다.

```
logging:
  ...
  llm:
    enable: false
    path: "/var/log/llm/llm.log"
    maxSize: 100000
    maxMinutes: 1440
    maxAge: 0
    maxBackups: 0
    compress: false
```

로깅 경로 변경

현재 버전에서는 설치 이후 로깅 경로 변경을 지원하지 않습니다.

메트릭

솔라 박스에서는 Prometheus에서 사용할 수 있는 기본 지표들을 /metrics API 를 통해서 제공합니다.

다음 경로를 통해 적재되는 지표들을 확인할 수 있습니다.

```
$> curl <NODE_IP>:8080/metrics
```

제공하는 지표는 다음과 같습니다.

- 요청 수: Earth_request_$(PATH){model=$(MODEL)}
- 응답 수
  - 성공 (2xx): Earth_success_$(PATH){model=$(MODEL)}
  - 에러 (4xx, 5xx): Earth_failure_$(PATH){model=$(MODEL), status_code=$(STATUS_CODE)}

- Latency:
  - Earth_latency_$(PATH){model=$(MODEL), units="milliseconds", quantile=$(QUANTILE)} (0.5, 0.9, 0.99)
  - Earth_latency_$(PATH)_sum{model=$(MODEL), units="milliseconds"}
  - Earth_latency_$(PATH)_count{model=$(MODEL), units="milliseconds"}
- (document-ocr, document-parse) Latency per page:
  - Earth_latency_per_page_$(PATH){model=$(MODEL), units="milliseconds", quantile=$(QUANTILE)} (0.5, 0.9, 0.99)
  - Earth_latency_per_page_$(PATH)_sum{model=$(MODEL), units="milliseconds"}
  - Earth_latency_per_page_$(PATH)_count{model=$(MODEL), units="milliseconds"}
- (document-ocr, document-parse) Confidence:
- (LLM) Request tokens:
  - Earth_prompt_tokens_$(PATH)_bucket{model=$(MODEL), units="tokens", le=$(VALUE)} (100, 500, 1000, 2000, +Inf)
  - Earth_prompt_tokens_$(PATH)_sum{model=$(MODEL), units="tokens"}
  - Earth_prompt_tokens_$(PATH)_count{model=$(MODEL), units="tokens"}

- (LLM) Response tokens:
  - Earth_completion_tokens_$(PATH)_bucket{model=$(MODEL), units="tokens", le=$(VALUE)} (100, 500, 1000, 2000, +Inf)
  - Earth_completion_tokens_$(PATH)_sum{model=$(MODEL), units="tokens"}
  - Earth_completion_tokens_$(PATH)_count{model=$(MODEL), units="tokens"}

⚠ _count 의 units는 N개(ea)로, 위 포맷에는 개별 데이터의 단위를 사용하는 버그가 있습니다.

⚠ quantile, bucket(le)은 구간과 계산 방법(누적)을 조정해야 합니다.

⚠ 모델명은 표준 명칭(예: ocr-123456)과 alias(예: ocr)의 통합이 되지 않고, 파라미터에 사용한 이름 그대로 집계합니다.

라이센스

솔라박스에서 API 로 서빙되는 모델들은 라이센스 등록을 통해서 정상적으로 API 를 사용할 수 있습니다. 프로젝트나 제품, 사용자 상황에 따라서 정식 제품 라이센스 또는 테스트 라이센스를 업스테이지로부터 발급 받아서 등록하면 API 를 정상적으로 호출 가능합니다.

라이센스 등록

라이센스는 Solar Box 가 실행되는 모든 노드에서 각각 등록 작업이 수행되어야 합니다. 등록 작업은 Earth 의 API 호출을 통해서 이루어 집니다.

```
$> curl -X POST <NODE_IP>:8080/api/license -d '{"key": "YOUR_LICENSE_KEY"}'
```

라이센스 조회

라이센스 조회 API 를 이용하면 등록된 라이센스 현황을 조회할 수 있습니다. 이 API 를 통해서 위 등록 작업이 정상적으로 이루어졌는지 다시 한번 확인합니다.

```
$> curl -X GET <NODE_IP>:8080/api/license
```

라이센스 만료와 갱신

라이센스 기간이 만료되면 API 동작이 정상적으로 수행되지 않습니다. 이 경우 라이센스 재발급과 갱신을 통해서 API 를 다시 사용할 수 있습니다. 라이센스 재발급은 수동으로 이루어 지는 과정이며 업스테이지 담당자로 부터 발급 받을 수 있습니다. 라이센스 만료 이전에 재발급을 통해 API 정상 동작을 유지해야 합니다.

라이센스 갱신은 등록 과정과 동일하게 진행합니다. 재발급 받은 라이센스 키를 등록 API 호출을 통해서 등록하면 라이센스 기간을 연장할 수 있습니다.

계정관리

솔라박스에서는 KeyCloak을 통해서 내부 시스템에 필요한 계정을 관리합니다. KeyCloak은 다음 주소로 접속할 수 있습니다.

```
http://<NODE_IP>:18080
```

기본 계정은 다음과 같습니다.

```
ID: admin
PW: admin
```

이 계정 관리 시스템에서는 솔라박스 부가기능 툴 (Admin tool, Monitoring 등)의 인증을 담당하고 있으며 사용자나 역할 등의 관리 작업을 수행할 수 있습니다.

사용자 추가하기

관리자는 다음과 같은 과정으로 사용자를 추가 할 수 있습니다.

1. KeyCloak 로그인
2. solarbox realm 선택
3. Users 선택
4. Add User 선택
5. 사용자 추가시 email verified on을 선택하고 Username, Email, First name, Last name을 모두 입력합니다.
6. Credentials 탭에서 Set password 누르고 사용할 비밀번호를 입력하고 temporary를 off 해줍니다.
7. 저장하여 사용자를 생성합니다.

역할 추가하기

관리자는 다음과 같은 과정으로 역할을 추가 할 수 있습니다.

1. KeyCloak 로그인
2. solarbox realm 선택
3. Realm Role 선택
4. Create role 선택
5. 원하는 역할 이름 입력 후 생성

사용자 - 역할 매핑

1. Users에서 사용자 선택
2. Role mapping 탭
3. Assign Role
4. Filter by realm roles 한 후 원하는 role을 부여해 줍니다.

Document Parse

이 장에서는 Document Parse 모델 추론 과정에 대해서 자세히 설명하고 성능 관련 제약 사항과 목표 성능을 달성하기 위한 아키텍처 구성에 필요한 정보를 제공합니다.

기본 제약사항

Document Parse 는 다음과 같은 제약 사항을 가집니다.

모델 추론 파이프라인

Document Parse 의 성능을 이해하기 위해서는 Document Parse 의 모델 추론 파이프라인에 대한 이해가 필요합니다. 이 섹션에서는 추론 과정에 사용되는 모델과 전체 추론 과정에 대해서 간단히 설명합니다.

문서 분리 및 변환

이 단계에서는 요청된 문서의 페이지를 분리하고 이미지로 변환합니다. 이 단계는 docmate 라는 서비스에서 수행 합니다. 이 단계에서는 AI 모델은 사용되지 않습니다.

OCR 또는 PDF 파싱

각 페이지의 단어와 각 단어의 bounding box 를 찾습니다. 이를 위해 OCR 을 수행하거나 PDF 파싱을 수행합니다. 이미지 입력인 경우 항상 OCR 을 수행하며 디지털 문서 입력인 경우 요청 파라미터가 ocr=force 인 경우에만 OCR 을 수행하며 나머지의 경우 PDF 파싱을 수행합니다.

OCR 은 딥러닝 모델이며, PDF 파싱은 파일 파싱을 통해서 수행됩니다. OCR 과 PDF 파싱 모두 결과는 동일하게 업스테이지 OCR 결과 포멧인 UFO 포멧으로 생성합니다.. 결과 포멧이 UFO 이기 때문에 내부적으로 PDF 파싱 모듈을 pdf2ufo 라고 부릅니다.

Layout Element Detection (LED)

이 단계에서는 딥러닝 모델이 페이지 내 모든 레이아웃 요소들을 검출합니다. 또한 각 레이아웃 요소들을 사람이 읽는 순서대로 정렬합니다.

Layout Table Recognition (LTR)

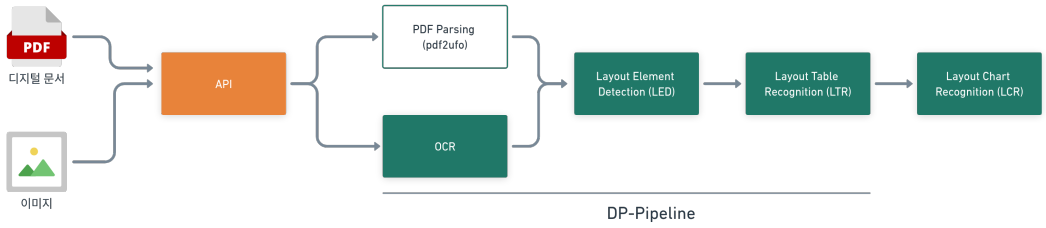
이 단계에서는 딥러닝 모델이 테이블 요소에 대해서 테이블 구조를 인식하여 HTML 형태로 변환합니다. LED 단계에서 테이블이 검출된 경우에만 모델 추론을 수행합니다.

Layout Chart Recognition (LCR)

이 단계에서는 딥러닝 모델이 차트 요소에 대해서 차트를 인식하여 테이블 형태로 변환합니다. LED 단계에서 차트 이미지가 검출된 경우에만 모델 추론을 수행합니다.

DP-Pipeline

OCR, LED, LTR 의 과정을 하나의 파이프라인으로 묶은 패키지 이름입니다. PDF 문서인 경우 OCR 을 수행하지 않기 때문에 PDF 파싱 결과를 입력으로 전달하면 파이프라인 내부에서 OCR 단계를 건너 뜁니다. DP-Pipeline 에 포함된 각 모델들은 별도로 학습되고 개발되지만 추론 과정에서 하나의 파이프라인으로 패키징되어 실제 서비스에 배포됩니다. DP-Pipeline 은 전체 추론 속도 향상 및 GPU 사용량 절감을 위해서 document-parse-250116 버전 부터 적용되었습니다.



문서 처리 파이프라인과 OCR 여부

Document Parse 는 다양한 모델이 파이프라이닝 되어 처리되며, 추론 속도에 가장 영향을 많이 미치는 것은 입력 문서에 대한 OCR 수행 여부 입니다. 요청시 ocr=force 를 명시하거나 이미지 파일을 입력으로 요청하게 되면 추론 시스템은 내부적으로 이 파일을 이미지로 변환하여 페이지 분리 후 각 페이지를 OCR 모델을 통해서 문자열을 추출하게 됩니다. OCR 의 경우 일반적으로 PDF 파싱보다 3배 이상 느리며 이미지의 특성에 따라서 성능 변화가 크기 때문에 OCR 수행 여부가 실제 추론 속도에 가장 큰 영향을 미칩니다.

다음은 AWS Sagemaker 에서 수행한 업스테이지 내부 벤치마크 테스트 결과의 일부입니다. OCR 을 수행하는 경우 PDF 파싱에 비해 페이지별 평균 수행 속도가 현저하게 느려지는것을 볼 수 있습니다.

Sagemaker Instance Type (Nvidia GPU type)	Testing duration	OCR or PDF Parsing	Total success responses	Average Latency per page	Pages Per Minute
ml.g5.2xlarge (A10G)	5 minutes	OCR	90	31.03 s	18
ml.g5.2xlarge (A10G)	5 minutes	PDF Parsing	328	8.85 s	65.6

OCR 수행시 중요한 부분은 이미지에 포함된 글자의 수 입니다. OCR 은 주어진 이미지에서 글자를 검출, 인식, 정렬하는 단계를 수행하기 때문에 글자가 많아질 수록 연산양이 비례해서 늘어나 추론 시간에 많은 영향을 미칩니다. 이 때문에 고객이 원하는 성능을 만족하기 위해서는 고객의 문서에 포함된 글자수가 어느정도 수준인지 이해가 필요합니다.

다음은 DP-Pipeline 에 테스트용 1페이지 문서를 문단의 개수와 글자의 개수를 조정하며 1분간 추론 테스트를 한 결과입니다. 결과에서 보듯 글자수가 늘어날 수록 분당 추론 횟수가 감소하는것을 볼 수 있

습니다. 참고로 진료비영수증 추론시간이 급격히 증가한 이유는 OCR 의 글자수도 영향이 있지만 테이블 등 전체적인 문서의 복잡도가 매우 높기 때문입니다.

테스트 문서	문단수	글자수	추론시간 (평균값)	추론시간 (p95)	1분 추론 건수
임의 작성 PDF	1	160	697ms	786ms	86
임의 작성 PDF	2	1300	830ms	943ms	73
임의 작성 PDF	3	2080	1.01s	1.2s	59
임의 작성 PDF	4	2261	1.03s	1.23s	58
진료비영수증	해당없음	3250	2.96s	3.94s	21

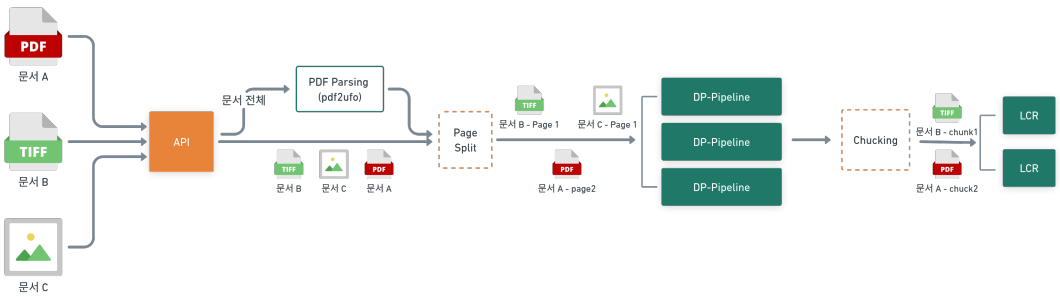
동시 요청과 병렬 처리 방식

API 서버가 여러개의 추론 요청을 동시에 받게 되면 각 요청의 문서는 청크 또는 페이지 단위로 분리되어 병렬적으로 추론이 수행됩니다. 이 때문에 동시 요청이 많은 경우 각 요청당 평균 처리 시간도 느려질 수 있습니다.

API 서버는 요청이 들어오면 OCR 을 수행할지 PDF 파싱을 수행할지 판단한 후 선택된 작업을 수행합니다. 이 때 PDF 파싱의 경우 문서 전체를 한번에 파싱합니다.

OCR 수행이 필요한 경우 곧바로 DP-Pipeline 으로 보내집니다. DP-Pipeline 은 페이지 한장 단위로 OCR-LED-LTR 추론을 수행하기 때문에 입력 문서를 페이지 단위로 분리하여 전달합니다. 만약 서버에 가용한 DP-Pipeline 이 5개가 떠 있다면 다섯 페이지가 각각 병렬적으로 추론 됩니다. 이때 다섯 페이지는 한 문서에서 온 페이지 일 수도 있고 각각 다른 요청에서 분리된 페이지 일 수 있습니다.

DP-Pipeline 수행 후 각각 처리된 페이지들은 동일 문서 안에서 하나의 청크로 묶여서 LCR 모델로 전달 됩니다. 청크 사이즈는 기본 5페이지 입니다. LCR 의 경우 전달된 입력에 대해서 차트가 검출될 때만 인식 작업을 수행하기 때문에 DP-Pipeline 과 동일한 개수의 모델이 떠 있을 필요는 없습니다.



API 는 유입되는 요청에 대해서 모델로 계속 요청을 전달하며 모델 서버는 요청을 받을 수 있을 때 까지 (즉, 시스템 오류가 발생하지 않는 한) 받아서 처리 합니다.

모델 개수와 처리량

이 섹션에서는 위에서 설명한 병렬 처리 내용을 기반으로 동일 요청에 대해서 실제 모델 개수와 처리량이 어떻게 유의미하게 변경되는지 테스트 결과를 제공합니다.

- 테스트 데이터: 13페이지 논문 PDF
- 테스트 시간: 2분

테이블 1. 처리 완료 요청량

동시 요청 개수 → DP-Pipeline 모델 개수 ↓	5	4	3	2	1
4	62	58	55	40	21
3	49	49	43	34	20
2	36	34	34	32	17
1	18	17	19	17	12

위 데이터에 의하면 동시 요청이 충분히 많은 경우 DP-Pipeline 의 개수가 늘어날수록 유의미하게 처리량도 늘어나는 것을 확인할 수 있습니다. 동시 요청이 5개인 경우 DP-Pipeline 이 1개인 경우보다 4개인 경우 3.4 배 정도의 처리량을 보였습니다.

테이블 2. 처리 완료 시간 (p95)

동시 요청 개수 → DP-Pipeline 모델 개수 ↓	5	4	3	2	1
4	13.52s	11.46s	8.82s	6.85s	6.17s
3	16.51s	12.92s	9.62s	8.52s	6.8s
2	23.41s	17.62s	13.74s	9.41s	8.47s
1	41.56s	32.99s	25.94s	17.52s	11.59s

위 데이터에 의하면 동시 요청이 충분히 많은 경우 DP-Pipeline 의 개수가 늘어날수록 유의미하게 처리시간도 줄어 드는것을 확인할 수 있습니다. 동시 요청이 5개인 경우 DP-Pipeline 이 1개인 경우보다 4개인 경우 3배 정도 빨라진 것을 보였습니다.

타임아웃

- Document Parse 전체 추론 요청 타임아웃 (문서 전체 단위): 5분
- 세부 타임아웃
  - PDF 파싱 요청 (문서 전체 단위): 3분
  - DP-Pipeline 모델 요청 (각 페이지 단위): 90초
  - LCR 모델 요청 (청크 단위): 90초

권장 시스템 사양

모델버전	세부 모델	CPU	RAM	GPU (VRAM)
document-parse-250116	DP-Pipeline	4 cpu	16 GB	7.5 GB
	Layout Chart Recognition	4 cpu	8 GB	4GB

FAQ

- 동시 요청된 문서는 하나씩 차례대로 처리되나요?
  - 아닙니다. 요청된 문서는 내부적으로 처리 단계에 따라서 페이지 단위로 분리되어 처리되거나 청크 단위로 분리되어 처리됩니다. API 에서 요청 파일 분리 후 가용한 모델에 페이지 또는 청크 단위로 요청을 하기 때문에 동시 요청이 들어온 경우 동시에 처리될 수 있습니다.
- 디지털 문서는 추론 과정에서 이미지로 변환되지 않나요?
  - 아닙니다. 일부 모델은 항상 이미지 입력을 필요로 하기 때문에 모든 문서는 API 에서 내부적으로 이미지로 변환하여 추론을 수행합니다.
- API 가 동시 처리할 수 있는 요청 수가 정해져 있나요?
  - 아닙니다. 현재 API 는 최대 동시 처리량이 정해져 있지 않고 (OOM 등의) 시스템 오류가 발생하지 않는 한 유입된 요청을 계속적으로 처리합니다.
- 100 페이지가 넘는 요청은 어떻게 되나요?
  - 현재 버전에서는 100 페이지 까지만 추론을 수행하여 응답을 생성합니다.
  - 100 페이지 이상의 문서를 처리해야 한다면, 파일을 100 페이지씩 잘라서 요청주셔야 합니다.
- CPU 와 Memory 는 추론 성능에 영향이 없나요?
  - 추론 과정에서 API 내부에서 페이지분리, 이미지 변환이나 일부 모델 추론 연산은 CPU 와

Memory 를 사용하기 때문에 리소스가 부족한 경우 이런 과정들이 병목이 될 수 있습니다. 특히 동시 처리가 많은 경우에 API 시스템의 컴퓨팅 리소스 사용량이 많아져 시스템 전반적인 응답 속도가 느려질 수 있습니다.